Active Demo Selection for VLA Fine-Tuning

Project tracking deck — LIBERO/SmolVLA pipeline

anh0001 | branch `research/conformal-active-demo-budgeting`

updated 2026-05-28

One-line goal: collect fewer robot demos, train a better policy.

What we are trying to solve, in plain terms

Setting. A pretrained robot policy (SmolVLA) is great at *many* tasks out of the box, but it still has to be fine-tuned for a *new* task using human-teleoperated demonstrations.

The pain. Each demo is expensive — a person physically teleops the robot through the task. In sim we get demos for free, but the lesson should transfer to real Piper arms where demos cost minutes of human time.

The question.

- You have a pool of $\sim$430 candidate demos (LIBERO-Spatial, 10 tasks).
- You have budget to train on **only 20**.
- **Which 20 should you pick?**

The hypothesis. Pick the demos the policy is *least sure* about $\Rightarrow$ same success rate at fewer demos.

The reality check. If active selection can't beat *random* on a strong pretrained model, the uncertainty signal is wrong — so far random is hard to beat, and that's the story this deck tracks.

What "input" and "output" actually mean here

INPUT

- A pretrained policy (SmolVLA-450M)
- A pool of $\sim$430 unlabeled candidate episodes
- A budget schedule: $N = 5, 10, 15, 20$
- A query method (the thing we compare)

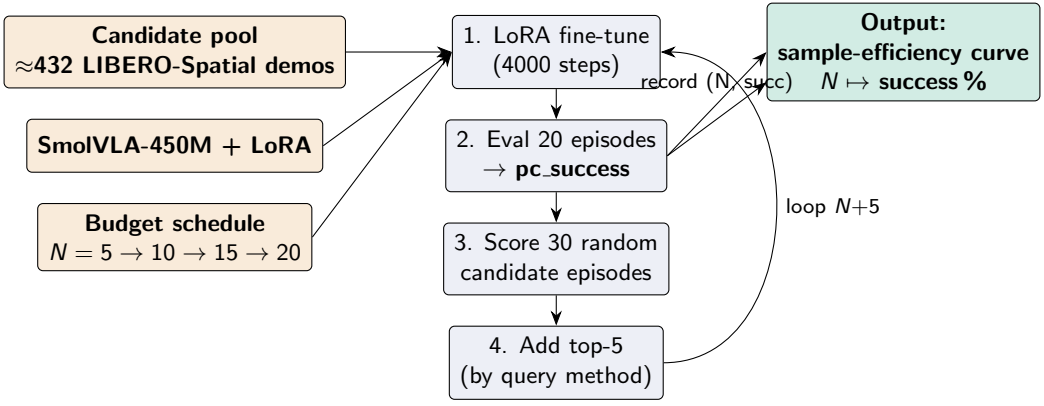
OUTPUT

- A *sample-efficiency curve*: success rate at each N.
- Plus a record of *which* episodes the method picked $\Rightarrow$ diagnostic for *why* it won or lost.

The query method is the only thing that changes between cells. Same backbone, same training recipe, same eval episodes. So if curves differ, the difference is the picking rule.

One "cell" = one (method, suite, seed). 4 training rounds per cell. $\approx$5 h wall-clock on one RTX 6000.

Pipeline at a glance



4 rounds per cell. One cell = (method, suite, seed). ≈5 h wall on one RTX 6000.

Pipeline glossary (what those words mean)

Episode One complete recorded demonstration of a single task attempt — a sequence of (camera images, robot state, action) tuples saved by a human teleop session. LIBERO-Spatial ships ∼430 such episodes across 10 tasks ("put X on Y" variants).

Candidate pool All episodes *not yet* in the current training budget. Each round, we pick from this pool.

Budget schedule $N{=}5 \rightarrow 10 \rightarrow 15 \rightarrow 20$ We don't train once at $N{=}20$. We train *four times*: first on 5 demos, then 10, then 15, then 20, adding 5 demos per round. The result is a curve $N \mapsto$ success %. This lets us see *when* the active method beats random, not just whether at one N.

Why subsample 30 random candidates? Scoring all ∼430 candidates needs ∼430 SmolVLA forward passes per round (or 4× that for dispersion). Subsampling 30 keeps cost bounded; randomness rotates coverage across rounds.

Why add top-5 each round? `demos_per_round=5`. Smaller increments would be invisible eval noise; larger increments would skip past the budgets where active selection matters.

`pc_success` *Percent-success* on the held-out eval suite: fraction of 20 LIBERO rollouts the trained policy completes, ×100. Range $[0, 100]$. This is the number we plot.

Query method 1: `random`

- **Input signal**: none.
- **Scoring rule**: score = `rng.random()` per candidate.
- **Selection rule**: top-5 by score (i.e. uniform pick of 5 from 30).
- **Expected failure mode**: bad demos by chance; high seed variance.
- **Why it matters**: the de-facto baseline; if a learner doesn't beat this on a well-pretrained VLA, the acquisition signal is anti-informative.

Query method 2: `conformal` [BROKEN]

- **Input signal**: teacher-forced policy loss per candidate episode (12 stride-8 frames, 1 forward pass each).
- **Scoring rule (as implemented)**: per candidate,
  1. add its own loss to the calibration buffer,
  2. return `loss/qhat` if buffer ≥ 20, else raw `loss`.
- **Selection rule**: top-5 by score.
- **Failure mode (confirmed)**: self-referential calibration + per-candidate threshold + sorted candidate order $\Rightarrow$ structurally picks *highest-indexed* episodes in the random subsample, not the most uncertain.
- See "Conformal bug" slide for the full diagnosis.

Query method 3: `entropy`

- **Input signal**: same teacher-forced loss as conformal.
- **Scoring rule**: score = mean loss over sampled frames.
- **Selection rule**: top-5 by score.
- **Expected failure mode**: loss is dominated by nuisance (trajectory length, contact noise, gripper switches), not epistemic uncertainty. Picks redundant "hard" demos.
- **Empirical**: trails random by 6-12 pp at N=20 (1 seed). Consistent with the nuisance hypothesis.

Query method 4: `dispersion`

- **Input signal**: K=4 samples from SmolVLA's flow-matching action head per frame; mean per-(step, dim) std across samples.
- **Scoring rule**: score = mean dispersion over sampled frames.
- **Selection rule**: top-5 by score.
- **Why this should help**: measures self-disagreement in *action space*, not next-token likelihood. On a saved checkpoint: dispersion stayed in $[0.037, 0.043]$ while teacher-forced loss spanned $[0.04, 8.69]$ across 4 episodes — loss is mostly nuisance.
- **Failure mode (also confirmed empirically)**: regresses at N=15 ($29 \rightarrow 22\,\%$). Without coverage, dispersion overselects from one visually-busy task.

Query method 5: `dispersion_quota`

- **Input signal**: same as `dispersion`.
- **Scoring rule**: same as `dispersion`.
- **Selection rule**: greedy round-robin under per-task coverage. For each pick, choose the candidate from the task with the smallest (budget + already-picked) count; break ties by score.
- **Why this should help**: forces task diversity; prevents the redundant-task collapse that pure dispersion shows.
- **Seed 0**: full curve, $N{=}20 \rightarrow$ **47.5 %** — beats every other method by 7–22 pp.
- **N=10 robustness stress test** (4 seeds, paired vs random): quota wins 3 of 4 seeds, mean +7.0 pp. Seed 1 (-9.5) is the lone outlier.
- **Current read**: per-task coverage is a real effect at $N{=}10$, not a one-seed fluke. Full $N{=}20$ curves now justified to confirm it carries to the full budget.

Current results: libero_spatial, max_demos = 20

N	rand s0	rand s1	conf s0	conf s1	ent s0	disp s0	d+q s0	disp s1	d+q s1
5	9.0	10.5	9.0	10.5	9.0	9.0	9.0	10.5	10.5
10	22.5	31.0	21.0	16.0	15.5	29.0	25.0	16.5	21.5
15	33.0	32.5	24.5	18.5	25.5	25.0	29.5	OOM	OOM
20	35.0	40.5	28.0	25.0	29.0	OOM	**47.5**	OOM	OOM

- Eval is *paired* across methods (env seed = 1000+ run seed). Gaps are not eval noise.
- d+q s0 = 47.5 at N=20 was the candidate win— but d+q s1 is under random at N=10 (21.5 vs 31.0). One seed up, one seed down.
- Grayed conformal columns: implementation broken; kept for reproducibility only.
- OOM: GPU co-tenant returned mid-train; partial curve only.

Conformal bug — how scoring went wrong

Codex's invariant check:
if conformal score = `loss/qhat` with a *global* `qhat`, ranking = entropy ranking.

Empirical: seed-0 $N{=}5{\to}10$ new picks

- conformal: $\{1394, 1447, 1458, 1493, 1523\}$
- entropy: $\{1394, 1458, 1567, 1574, 1579\}$

Overlap 2/5. Invariant violated.

Smoking gun in `active_loop.py:77{82`:

- Each candidate's own loss is added to the buffer *before* it is scored (self-reference).
- Buffer resets every round; `calib_min`$= 20$, `max_candidates`$= 30$.
- Candidates $0{-}19$ return raw loss ($\sim$0.03); $20{-}29$ return `loss/qhat` ($\sim$1.0). $20\times$ scale gap.
- Top-5 always lands in the last $\sim$10 iterated candidates = highest-indexed in the sorted subsample.

Implication: conformal in this top-k pool setting reduces to a monotone transform; ranking $\equiv$ entropy. The branding adds no acquisition signal.

Status

Done

- PS.4 single-task gate passed (80 % on libero_spatial task 0).
- 5 methods on libero_spatial; conformal bug found (rank-invariant violated).
- Pure `dispersion` regresses at N=15 reproducibly (both seeds, both runs).
- `dispersion_quota` s0 hit 47.5 % at N=20, but s1 is under random at N=10.

Open question

- Is `dispersion_quota` a *robust* low-budget learner, or was 47.5 % a one-seed fluke?

Recurring blocker

- A co-tenant (HMDB51 ConvGRU) keeps grabbing both RTX 6000s mid-run and OOM-killing the N=15–20 rounds. 3 cells lost this way so far.

N=10 stress test (codex-recommended): quota passed

Design: fixed N=10, `random` vs `dispersion_quota`, 4 seeds, paired demos + paired eval. Decision rule set *before* looking at the data.

seed	random	quota	Δ
0	22.5	25.0	+2.5
1	31.0	21.5	−9.5
2	28.5	36.0	+7.5
3	12.5	40.0	+27.5
mean	23.6	30.6	**+7.0**

Verdict: quota wins 3 of 4 seeds; seed 1 is the lone outlier. Rule says: *run full* N=20 *curves for quota*. Caveat: N=10 confirms robustness, not yet the N=20 magnitude.

Aside: the test cost 8 cells — a recurring co-tenant (HMDB51) OOM-killed the N=10 rounds 4×; the clean pairs came from running one cell at a time on whichever GPU was momentarily free.

The honest paper, if the method win doesn't hold

The fallback is *not* a failed project — it is a sharper, more defensible claim:

> *For low-demo VLA fine-tuning, acquisition signals that look principled under supervised active learning become self-referential (conformal) or anti-informative (teacher-forced loss, action dispersion). Under paired demos and paired eval, they fail to robustly beat random; per-task coverage gives single-seed gains that don't generalize. The contribution is a diagnostic account of why fixed-budget pool AL for VLAs is brittle, not a new selector.*

Minimum artifact: (1) conformal self-reference bug + rank invariant; (2) reproducible $N{=}15$ regression of pure dispersion; (3) random-vs-quota table over ≥ 2 seeds, *labelled as robustness evidence*, not a win.